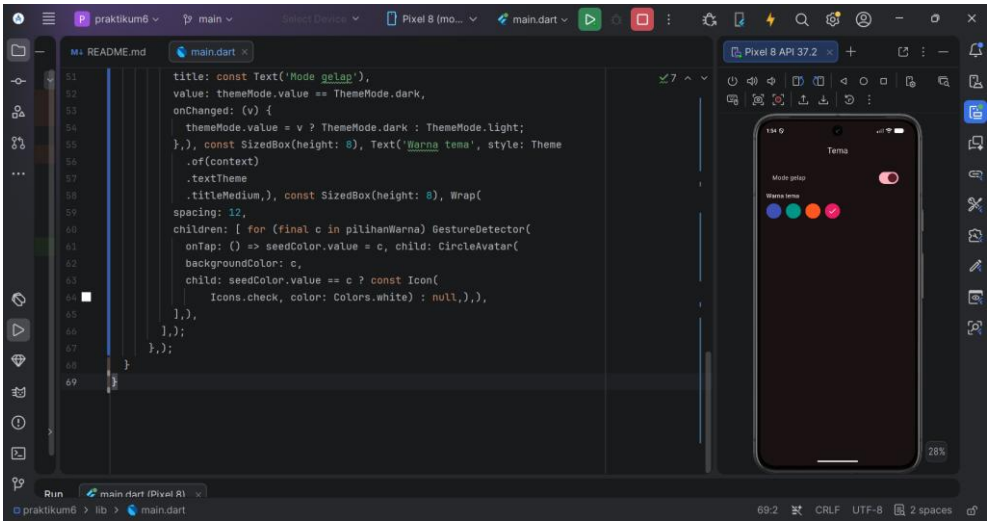
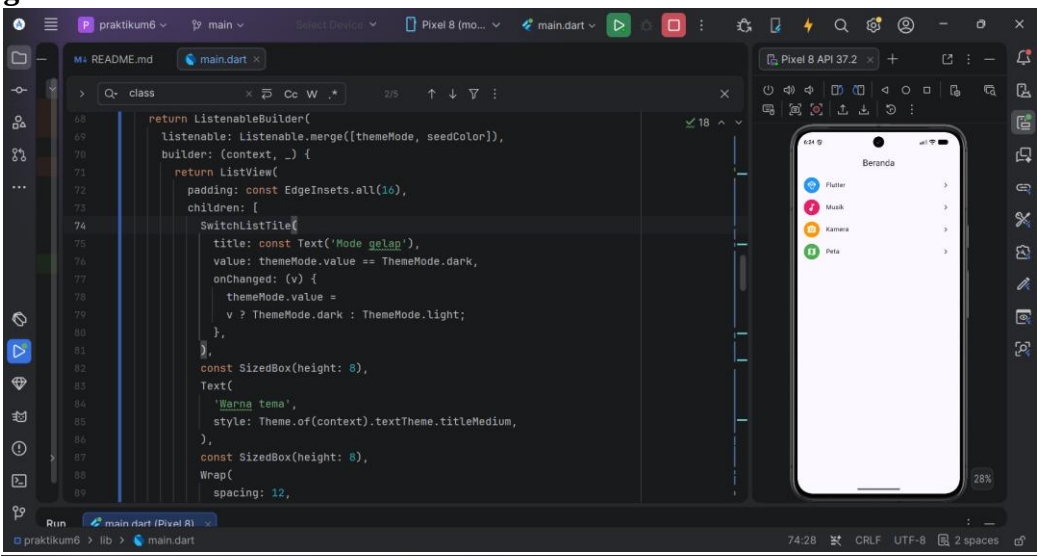
DOKUMENTASI MODUL 6

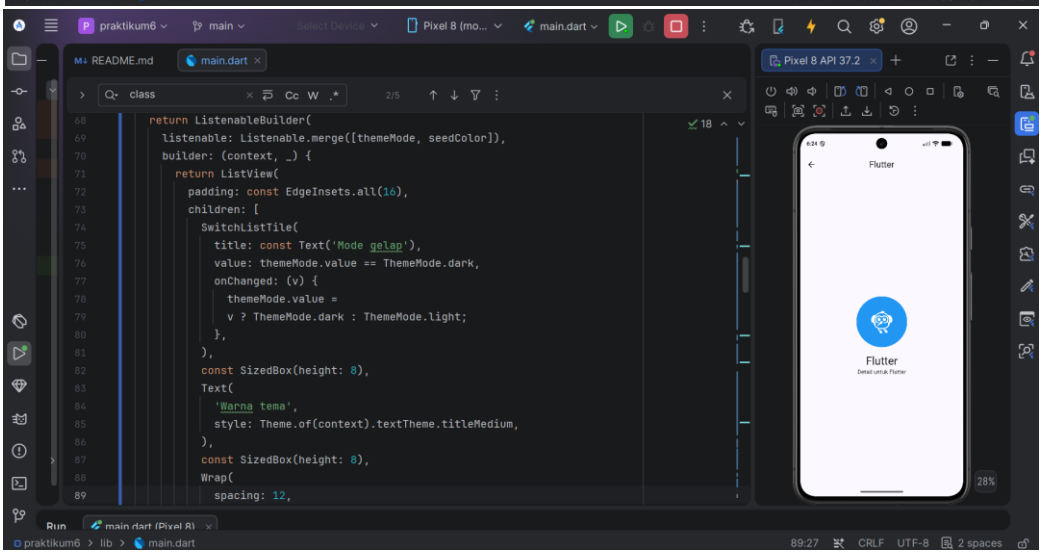
Praktikum dan Latihan Mandiri

1. Bagian A

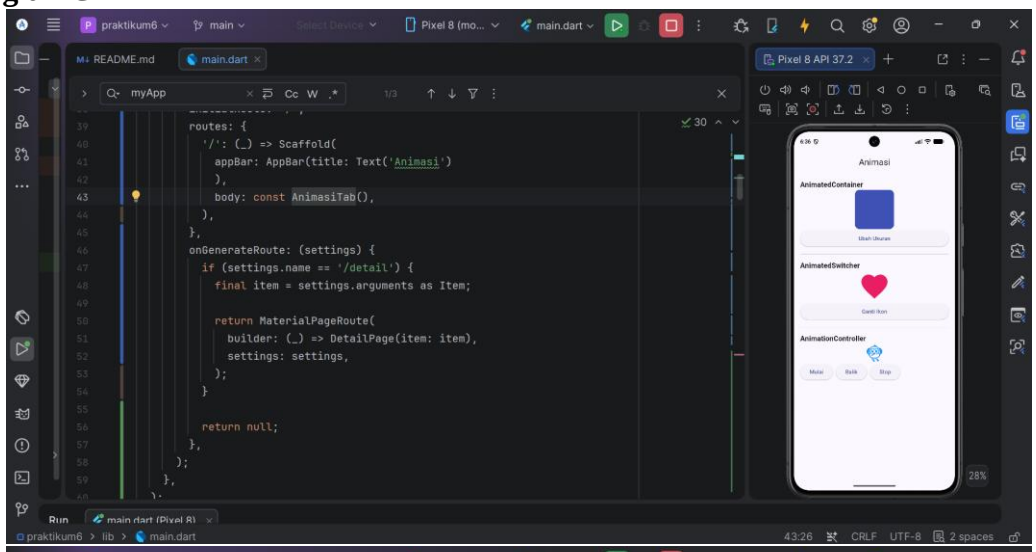


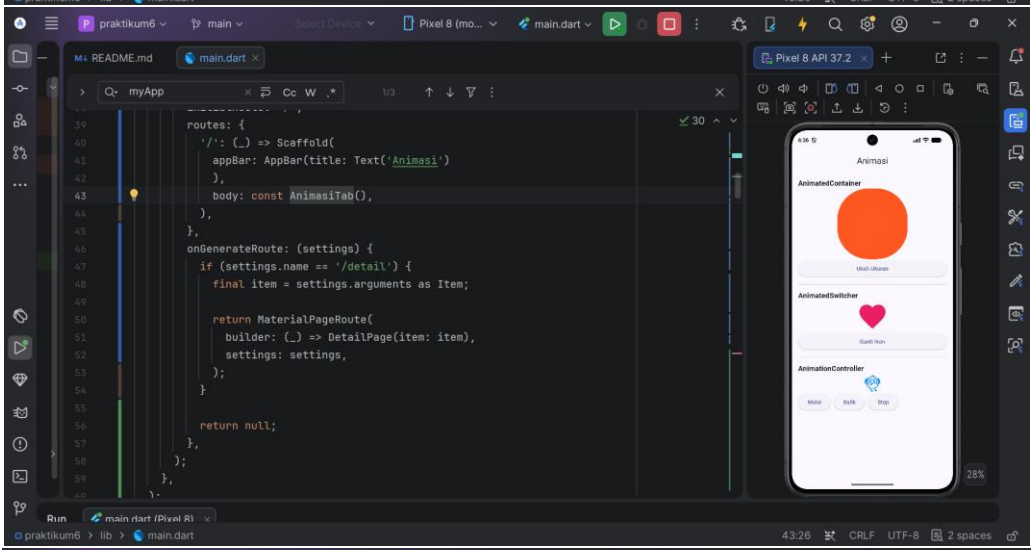
2. Bagian B

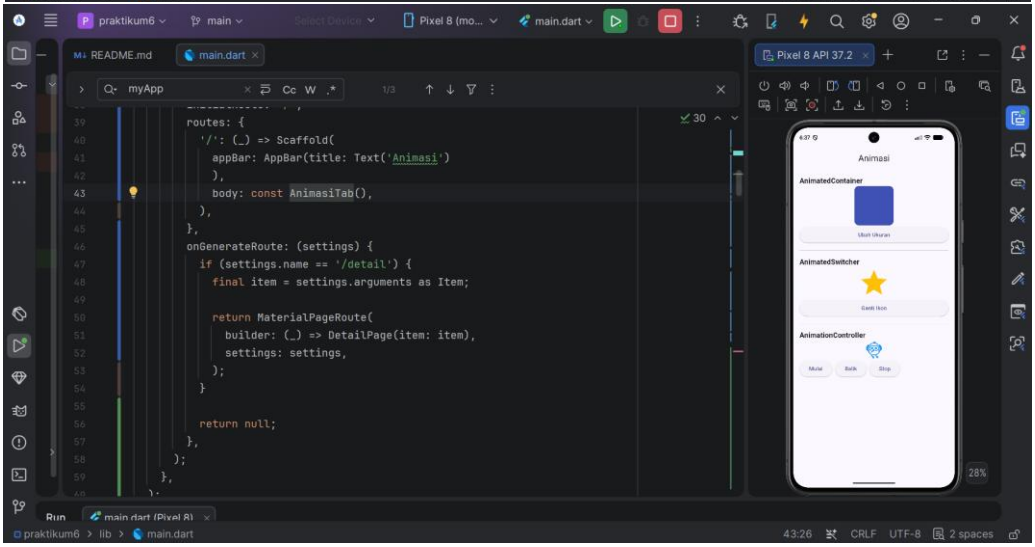


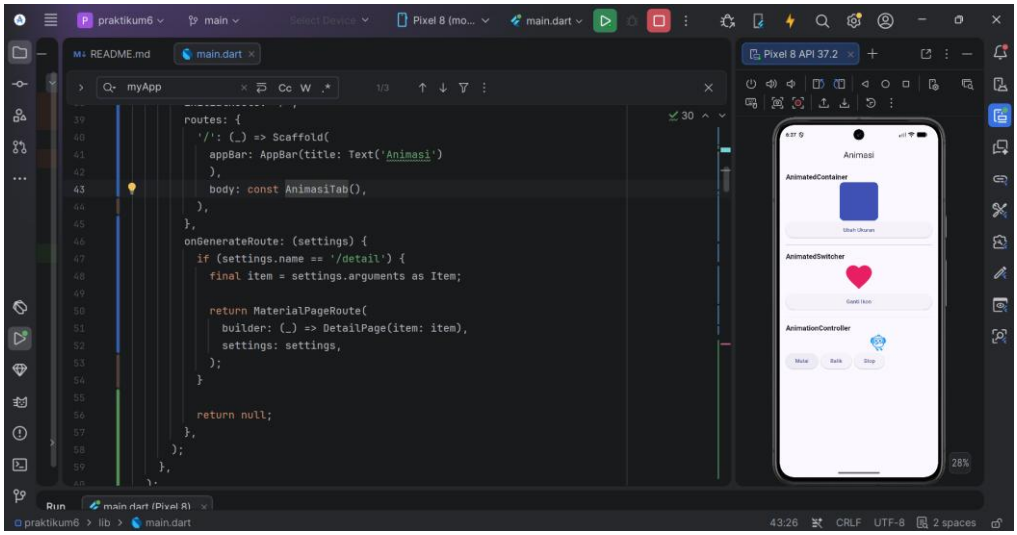


3. Bagian C

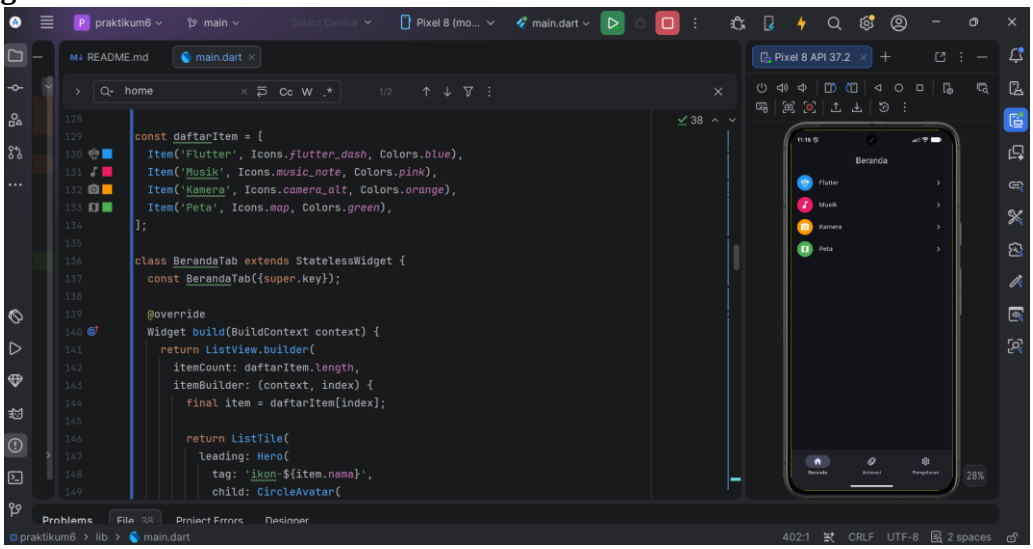

praktikum6
main
Pixel 8 (mo...
main.dart
README.md
main.dart
myApp
Pixel 8 API 37.2
routes: {
'/': (_) => Scaffold(
appBar: AppBar(title: Text('Animasi')
),
body: const AnimasiTab(),
),
},
onGenerateRoute: (settings) {
if (settings.name == '/detail') {
final item = settings.arguments as Item;
return MaterialPageRoute(
builder: (_) => DetailPage(item: item),
settings: settings,
);
}
return null;
},
);
},
Animasi
AnimatedContainer
AnimatedSwitcher
AnimationController
Run
main.dart (Pixel 8)
43:26
CRLF
UTF-8
2 spaces


praktikum6
main
Pixel 8 (mo...
main.dart
README.md
main.dart
myApp
Pixel 8 API 37.2
routes: {
'/': (_) => Scaffold(
appBar: AppBar(title: Text('Animasi')
),
body: const AnimasiTab(),
),
},
onGenerateRoute: (settings) {
if (settings.name == '/detail') {
final item = settings.arguments as Item;
return MaterialPageRoute(
builder: (_) => DetailPage(item: item),
settings: settings,
);
}
return null;
},
);
},
Animasi
AnimatedContainer
AnimatedSwitcher
AnimationController
Run
main.dart (Pixel 8)
43:26
CRLF
UTF-8
2 spaces

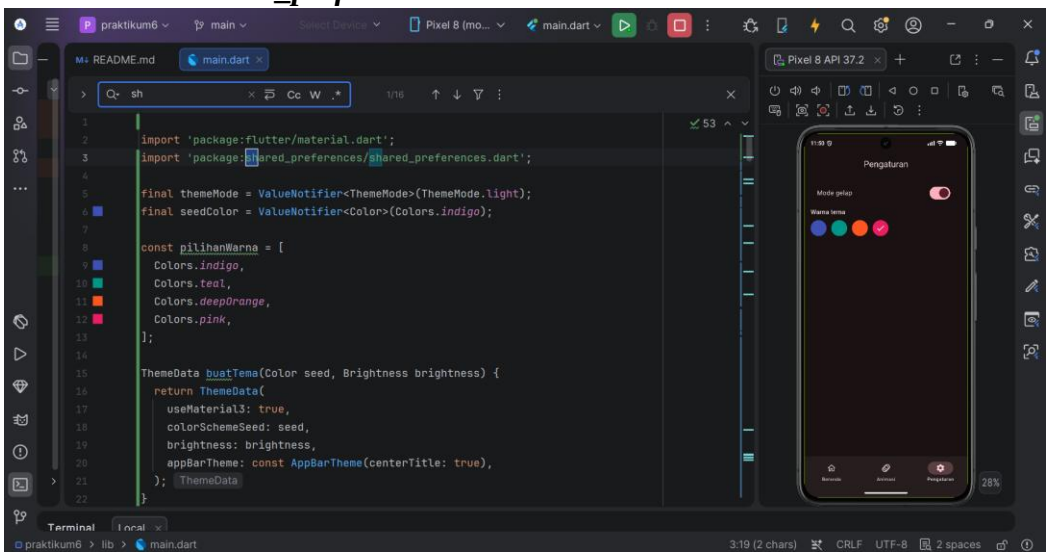

praktikum6
main
Pixel 8 (mo...
main.dart
README.md
main.dart
myApp
Pixel 8 API 37.2
routes: {
'/': (_) => Scaffold(
appBar: AppBar(title: Text('Animasi')
),
body: const AnimasiTab(),
),
},
onGenerateRoute: (settings) {
if (settings.name == '/detail') {
final item = settings.arguments as Item;
return MaterialPageRoute(
builder: (_) => DetailPage(item: item),
settings: settings,
);
}
return null;
},
);
},
Animasi
AnimatedContainer
AnimatedSwitcher
AnimationController
Run
main.dart (Pixel 8)
43:26
CRLF
UTF-8
2 spaces



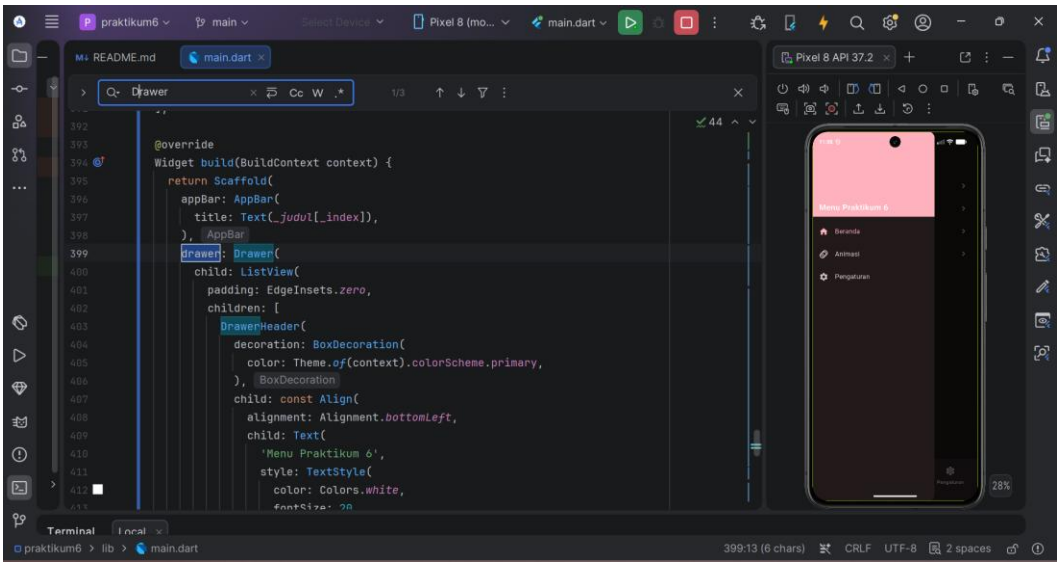
4. **Bagian D**



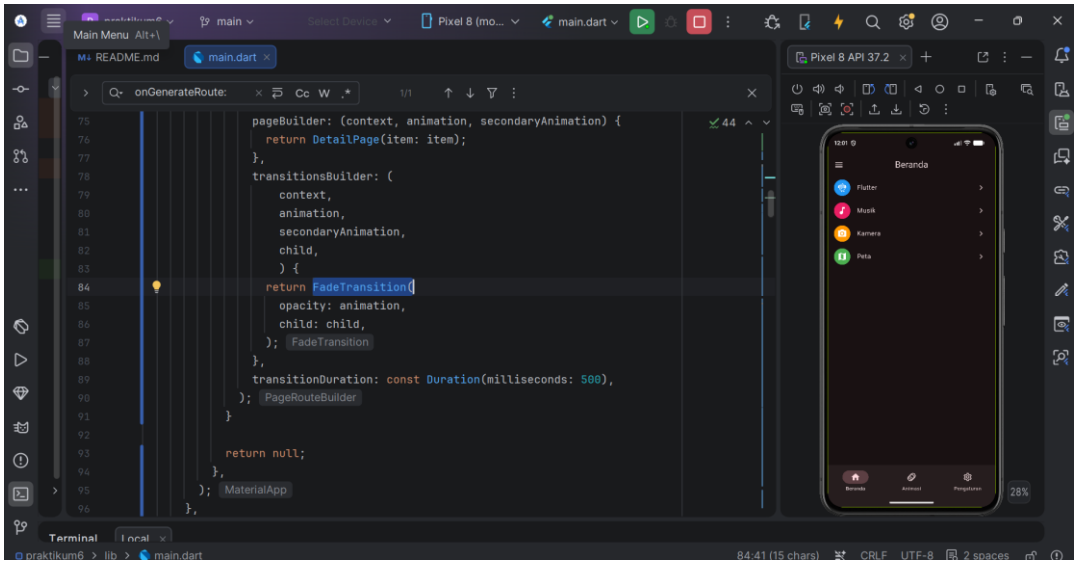
5. **Menambahkan *shared_preferences***



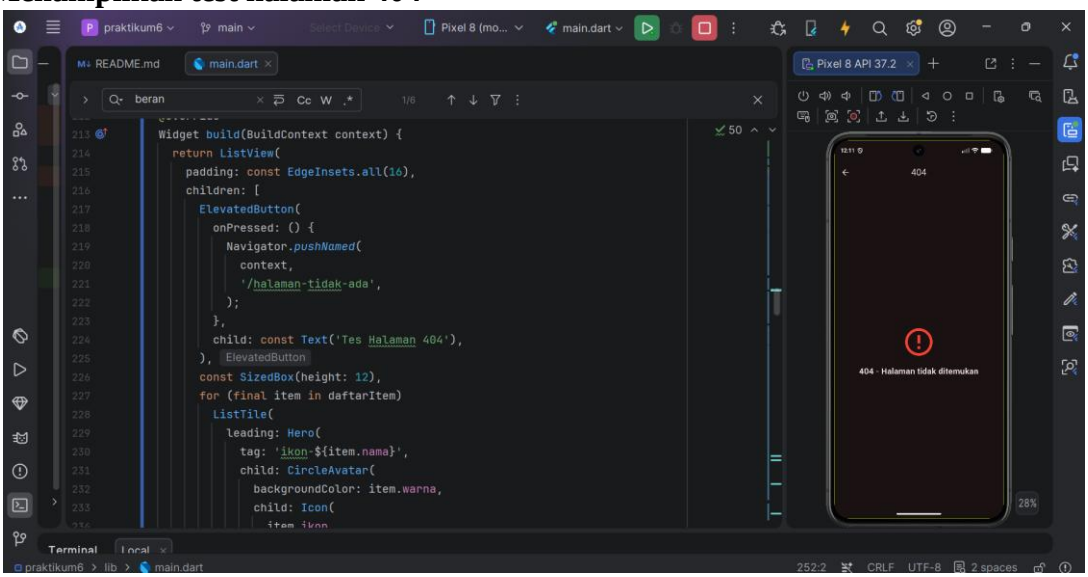
6. **Menambahkan *Drawer***



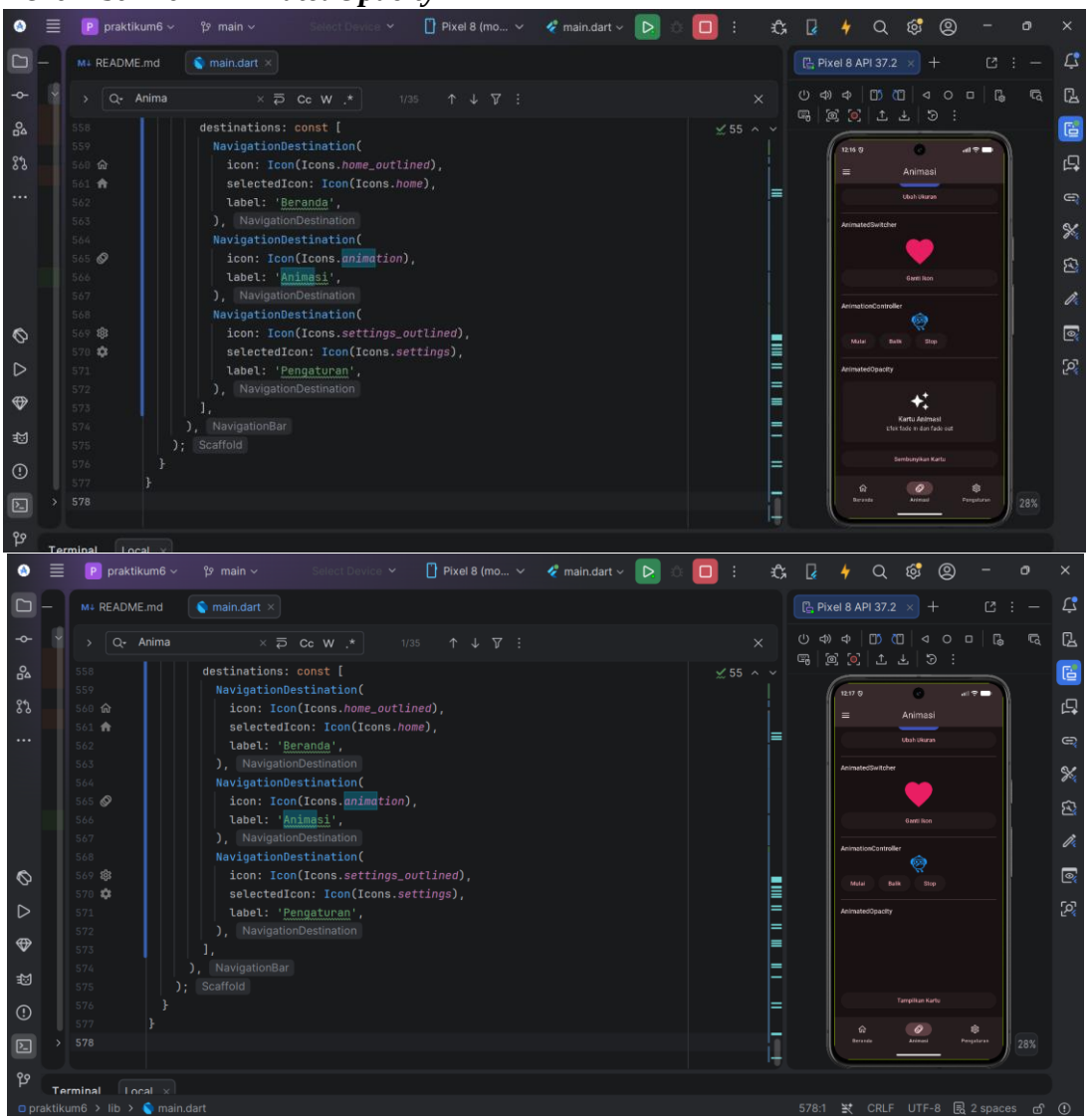
7. **Menambahkan efek *FadeTransition***



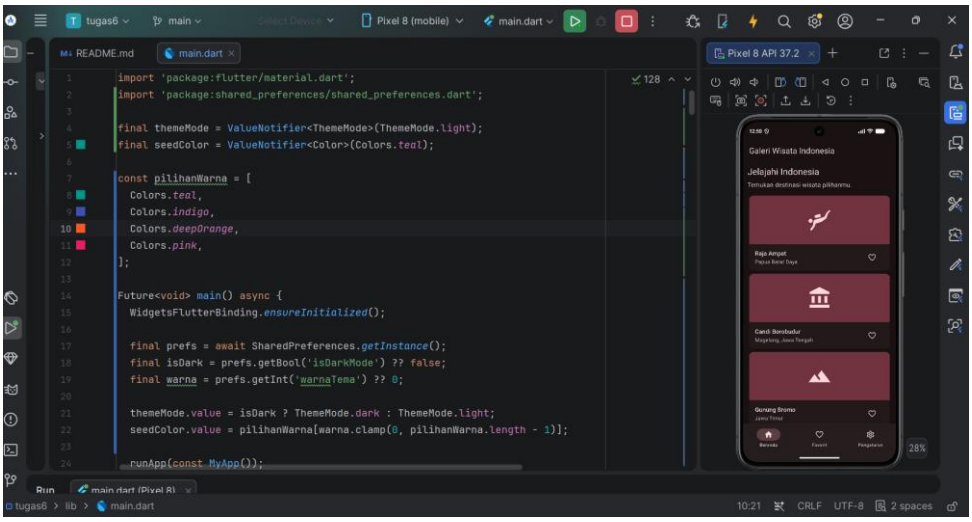
8. **Menampilkan test halaman 404**

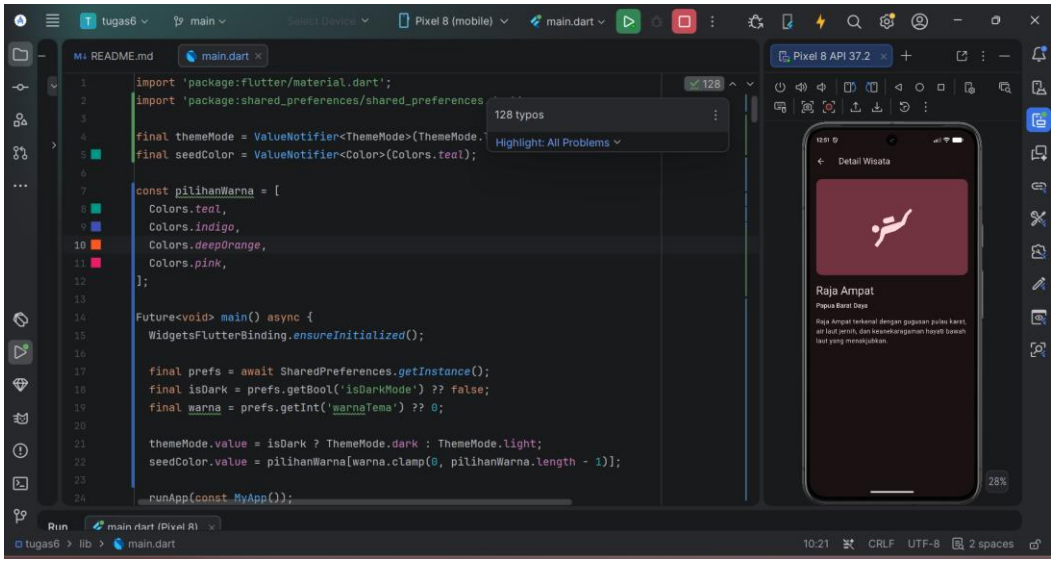


9. Menambahkan *AnimatedOpacity*



Tugas






```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

final themeMode = ValueNotifier<ThemeMode>(ThemeMode.light);
final seedColor = ValueNotifier<Color>(Colors.teal);

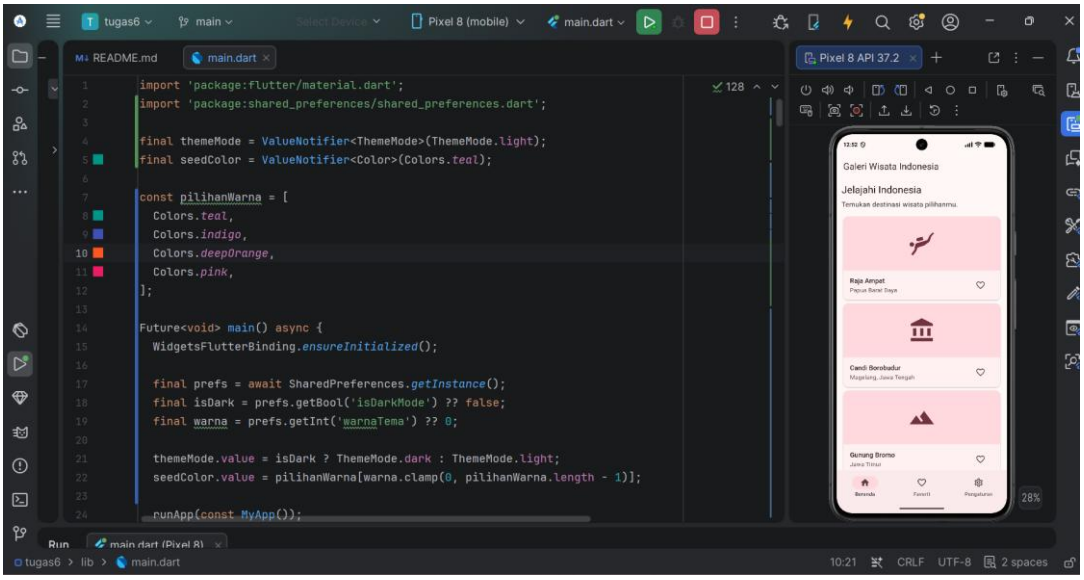
const pilihanWarna = [
  Colors.teal,
  Colors.indigo,
  Colors.deepOrange,
  Colors.pink,
];

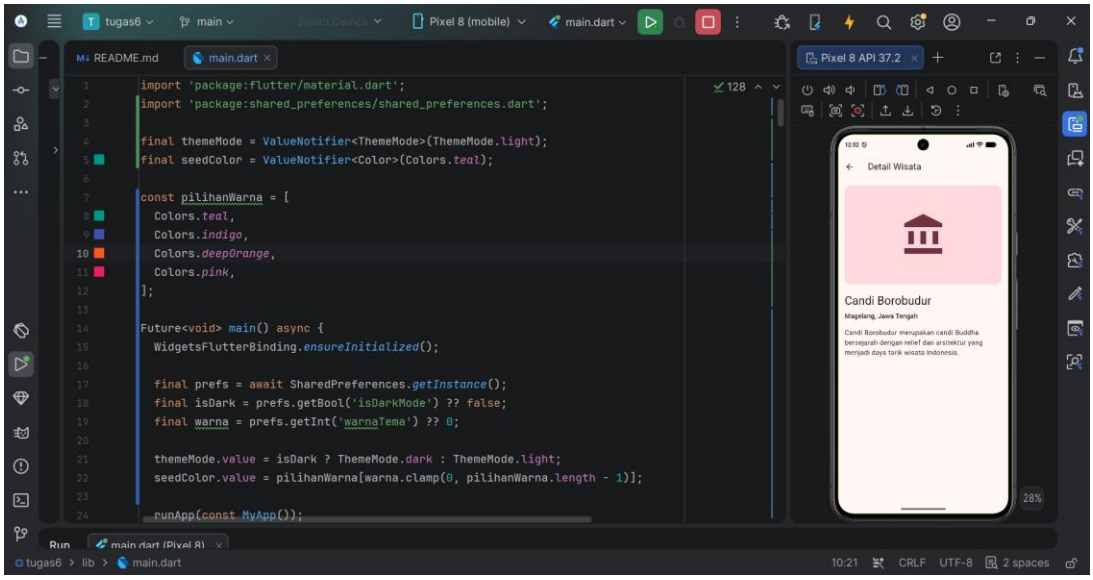
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

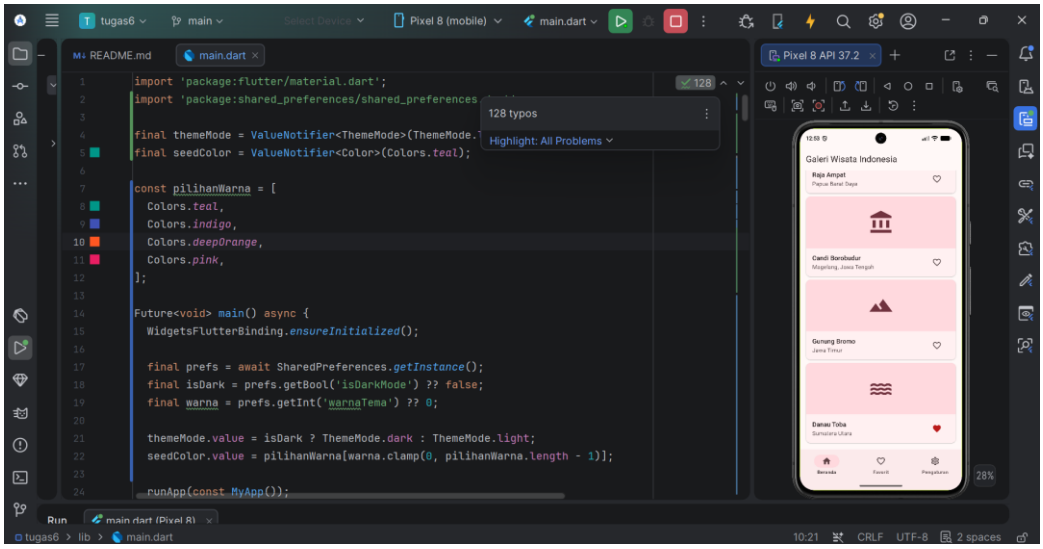
  final prefs = await SharedPreferences.getInstance();
  final isDark = prefs.getBool('isDarkMode') ?? false;
  final warna = prefs.getInt('warnaTema') ?? 0;

  themeMode.value = isDark ? ThemeMode.dark : ThemeMode.light;
  seedColor.value = pilihanWarna[warna.clamp(0, pilihanWarna.length - 1)];

  runApp(const MyApp());
```








```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

final themeMode = ValueNotifier<ThemeMode>(ThemeMode.light);
final seedColor = ValueNotifier<Color>(Colors.teal);

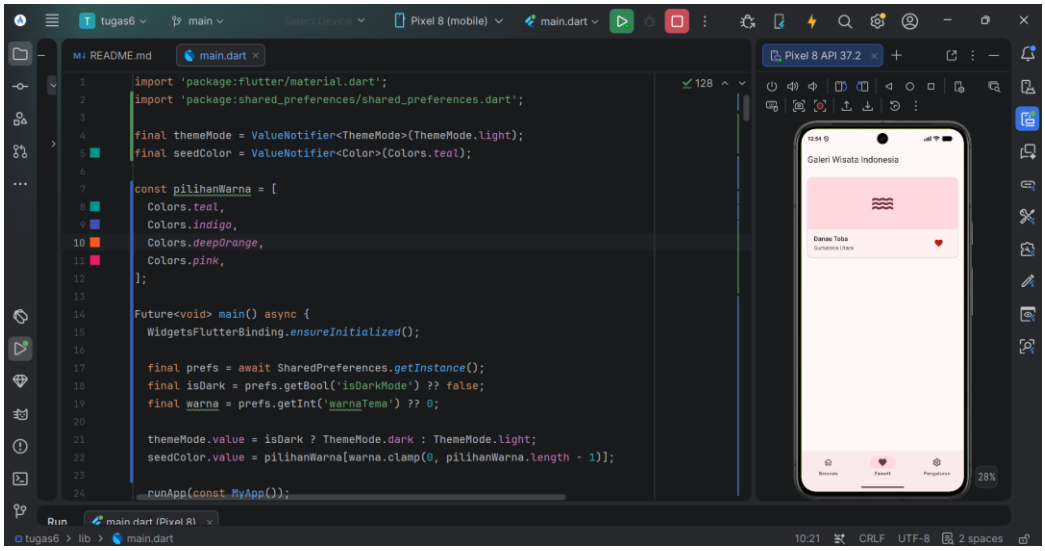
const pilihanWarna = [
  Colors.teal,
  Colors.indigo,
  Colors.deepOrange,
  Colors.pink,
];

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  final prefs = await SharedPreferences.getInstance();
  final isDark = prefs.getBool('isDarkMode') ?? false;
  final warna = prefs.getInt('warnaTema') ?? 0;

  themeMode.value = isDark ? ThemeMode.dark : ThemeMode.light;
  seedColor.value = pilihanWarna[warna.clamp(0, pilihanWarna.length - 1)];

  runApp(const MyApp());
```




```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

final themeMode = ValueNotifier<ThemeMode>(ThemeMode.light);
final seedColor = ValueNotifier<Color>(Colors.teal);

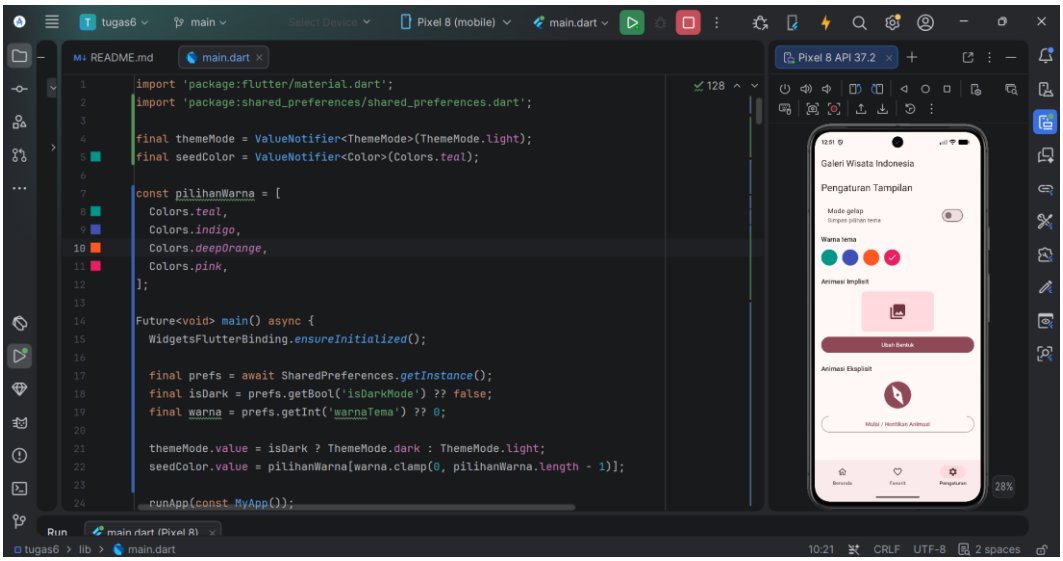
const pilihanWarna = [
  Colors.teal,
  Colors.indigo,
  Colors.deepOrange,
  Colors.pink,
];

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  final prefs = await SharedPreferences.getInstance();
  final isDark = prefs.getBool('isDarkMode') ?? false;
  final warna = prefs.getInt('warnaTema') ?? 0;

  themeMode.value = isDark ? ThemeMode.dark : ThemeMode.light;
  seedColor.value = pilihanWarna[warna.clamp(0, pilihanWarna.length - 1)];

  runApp(const MyApp());
```




```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

final themeMode = ValueNotifier<ThemeMode>(ThemeMode.light);
final seedColor = ValueNotifier<Color>(Colors.teal);

const pilihanWarna = [
  Colors.teal,
  Colors.indigo,
  Colors.deepOrange,
  Colors.pink,
];

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  final prefs = await SharedPreferences.getInstance();
  final isDark = prefs.getBool('isDarkMode') ?? false;
  final warna = prefs.getInt('warnaTema') ?? 0;

  themeMode.value = isDark ? ThemeMode.dark : ThemeMode.light;
  seedColor.value = pilihanWarna[warna.clamp(0, pilihanWarna.length - 1)];

  runApp(const MyApp());
```

Pertanyaan Refleksi

1. **Apa perbedaan animasi implisit dan eksplisit? Kapan memilih masing-masing?**
   Jawab: Animasi implisit adalah animasi yang berjalan otomatis ketika nilai properti berubah, contohnya AnimatedContainer. Animasi eksplisit menggunakan AnimationController untuk mengatur durasi, kecepatan, dan pengulangan animasi. Animasi implisit dipilih untuk perubahan sederhana, sedangkan animasi eksplisit dipilih untuk animasi yang membutuhkan kontrol lebih detail.

2. **Mengapa dua Hero harus memiliki tag yang sama, dan apa yang terjadi bila tag-nya muncul dua kali di halaman yang sama?**
   Jawab: Dua widget Hero harus memiliki tag yang sama agar Flutter dapat menghubungkan widget asal dan tujuan untuk menghasilkan transisi animasi antarhalaman. Jika terdapat dua widget Hero dengan tag yang sama dalam satu halaman dan subtree yang sama, Flutter dapat menampilkan error karena tag Hero harus unik dalam konteks tersebut.

3. **Apa keuntungan memakai Theme.of(context) dibanding menulis warna langsung pada widget?**
   Jawab: *Theme.of(context)* memungkinkan widget mengikuti tema aplikasi secara konsisten, termasuk mode terang dan gelap. Penggunaan tema juga mempermudah perubahan tampilan secara terpusat dan mengurangi penggunaan warna hard-coded.

4. **Apa perbedaan berpindah tab dengan IndexedStack dan membuka halaman dengan Navigator?**
   Jawab: *IndexedStack* digunakan untuk berpindah tab sambil mempertahankan state setiap tab. Sementara itu, Navigator digunakan untuk membuka atau menumpuk halaman baru dalam riwayat navigasi, misalnya membuka halaman detail dari halaman utama.

5. **Mengapa AnimationController wajib di-dispose?**
   Jawab: *AnimationController* wajib di-dispose ketika widget tidak lagi digunakan agar sumber daya yang dipakai dapat dilepaskan dan animasi berhenti berjalan. Hal ini mencegah kebocoran sumber daya serta menghindari masalah ketika controller masih berjalan setelah widget dihapus.